LogShield: In-Pod Secret Sanitization and Autonomous Closed-Loop Containment for Cloud-Native Microservices

Rounak Neema
Dept. of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
rounak.neema046@nmims.in

Manas Borse
Dept. of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
manas.borse057@nmims.in

Arnab Chowdhury
Dept. of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
arnab.chowdhury032@nmims.in

Prince Roy
Dept. of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
prince.roy031@nmims.in

Rudrani Ingle
Dept. of Computer Engineering
MPSTME, NMIMS University
Mumbai, India
Rudrani.Ingle@nmims.edu

***Abstract*—Microservice architectures managed by Kubernetes clusters generate huge volumes of container telemetry. Under continuous delivery pressures, developers often emit confidential credentials into standard output streams, including database connection strings, cloud access keys, private keys, and OAuth headers (CWE-532). Static scanners cannot inspect dynamically assembled runtime strings, and existing defenses leave major gaps. Centralized cloud sanitizers require network egress of raw log lines, exposing organizations to transit interception and violating data privacy regulations such as GDPR and HIPAA, while passive log redactors mask text after the fact but leave compromised credentials active in production environments until manual triage occurs hours or days later. To tackle these vulnerabilities, we present LogShield, a Kubernetes sidecar framework for real-time secret sanitization, keyed privacy-preserving lineage tracking, and autonomous closed-loop containment within the pod boundary. Masked credentials are encrypted locally in the SecureReveal Vault using Fernet authenticated encryption (AES-128-CBC with HMAC-SHA256), paired with a tamper-evident SHA-256 hash-chained audit ledger with remote SIEM anchoring. LogShield combines an O(N) Aho-Corasick prefilter, an adaptive sliding-window token fusion engine that defeats delimiter-split and hex-escaped obfuscation, a 6-factor Secret Confidence Score (SCS) model, and a tri-state decision gate; when active leaks emerge, it appraises blast radius and enforces automated mitigation via dynamic Kubernetes V1NetworkPolicy egress isolation and cloud credential rotation webhooks. To track cross-service credential sprawl without sharing plaintext strings, LogShield applies canonical keyed blind HMAC-SHA256 fingerprinting to join multi-representation artifacts (raw, Base64, Hex, URL-encoded), supported by a scoped exemption engine for pre-registered credentials. LogShield is evaluated across four experimental corpora, an isolated Kubernetes control plane testbed, and an end-to-end 5-microservice cluster. In standard ablation testing (N = 9,205), LogShield achieves an F1-score of 0.9997 (precision 1.0000, recall 0.9993) with zero false positives on benign controls (FPR ≤ 0.063%). On a balanced 2,400-sample adversarial evasion benchmark, Token Fusion raises detection recall from 32.20% to 91.30%, lowering evasion bypasses from 67.80% to 8.70% (a 59.1 percentage-point reduction) while producing no false positives on benign high-entropy system tokens. In separate statistical validation on 20,000 held-out entries, the ML-trained normalized model detects 228 additional secrets over domain-expert heuristic weights ($\chi 2 = 226.00$, $p = 4.43 \times 10{-}51$) at a strictly zero false-positive threshold (T = 80). LogShield achieves automated workload egress quarantine with 16.7–26.7 ms policy dispatch latency and validated zero collateral disruption on non-leaking workloads in live Kubernetes testing.**

***Index Terms*—Cloud-native security, Kubernetes sidecar, log sanitization, secret detection, CWE-532, Shannon entropy, Aho-Corasick automaton, NetworkPolicy quarantine, secret lineage.**

I. INTRODUCTION

Containerized microservices managed by Kubernetes form the primary foundation for cloud-native software systems [6]; within these clusters, application logging offers the central source of telemetry for operational observability, distributed debugging, and security monitoring [25], [2]. Large-scale cloud deployments routinely create terabytes of unstructured application output each day [5].

High logging volumes create persistent security risks. Under rapid deployment schedules and complex runtime debugging requirements, developers regularly emit data structures containing raw credentials [20], [3]. Prevalent examples include logging complete HTTP headers, dumping environment variables during exceptions, and printing database URIs during connection timeouts:

Listing 1: Representative Credential Leakage in Application Logs

```
logger.debug(f"DB URI: {DATABASE_URL}")
# postgres://admin:MyP@ssw0rd@prod-db:5432/app
```

```
logger.error(f"Auth error: {headers['Authorization']}")
# Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
logger.info(f"Payment token: {STRIPE_SECRET_KEY}")
# sk_live_51H7x8...
logger.debug(f"Environment: {os.environ}")
# AWS_SECRET_ACCESS_KEY=AKIAIOSFODNN7EXAMPLE...
```

When emitted to standard output (stdout), these strings pass directly to node-level forwarding daemons (such as Fluent Bit or Vector) and travel across network perimeters to centralized log platforms (such as Elasticsearch, Grafana Loki, or Datadog) [22]. Once indexed, credentials are stored in plain text, accessible to anyone with log read permissions [14]. This vulnerability corresponds to CWE-532 (Insertion of Sensitive Information into Log File) [17] and CWE-798 (Use of Hard-coded Credentials) [15].

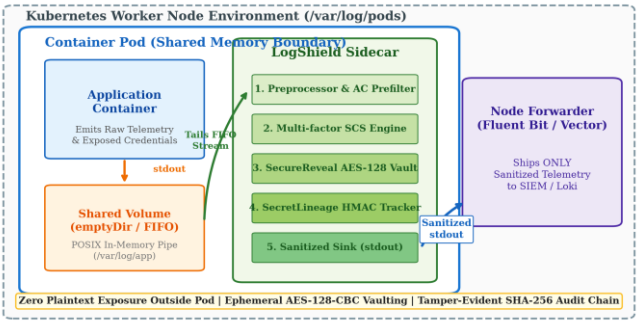


Fig. 1. LogShield in-pod deployment. Co-located within the Kubernetes pod, the sidecar intercepts raw logs over a shared memory-backed emptyDir (tmpfs), detects secrets with adaptive token fusion, isolates them in an encrypted vault, enforces autonomous egress quarantine, and emits sanitized logs to container stdout.

Zhi et al. [20] analyzed 413 real-world vulnerabilities reported in the National Vulnerability Database (NVD) under CWE-532 and found that 67.8% were exploitable across network perimeters, with 89.3% requiring low attacker effort. In 2022 alone, GitGuardian detected over 10 million leaked credentials in public repositories, a 67% increase year-over-year [12].

A. Shortcomings of Current Countermeasures

Existing security controls fail to confront runtime container log streams because of architectural drawbacks:

Static Code Scanners: Tools such as Gitleaks [10], TruffleHog [4], and detect-secrets [9] assess static source files and commit histories; they cannot inspect dynamic runtime strings produced during database transactions, runtime token grants, or third-party API calls.

Secrets Managers: Platforms such as HashiCorp Vault or AWS Secrets Manager inject credentials securely into container memory. However, they do not monitor whether an application later prints those credentials into log streams.

Cloud-Side Log Sanitizers: Scanners such as Datadog Sensitive Data Scanner or Google Cloud DLP operate downstream in centralized storage; credentials must leave the container and cross the network before inspection occurs, introducing transit exposure and violating strict data residency requirements under GDPR Article 25 [24], HIPAA 45 CFR Part 164 [32], PCI-DSS v4.0 Requirement 8 [18], and NIST SP 800-53 controls SC-28 and AU-9 [21].

Single-Factor Heuristic Scanners: Runtime filters relying on regular expressions or Shannon entropy alone create high false-positive rates (>15%), mistaking benign UUIDv4 strings, MD5 ETags, and commit hashes for credentials [19].

Passive Redaction Limitations: Standard redactors simply replace strings with mask tokens; they do not assess multi-service blast radius, link obfuscated representations, or reduce live credential compromise, leaving active credentials exposed until human security teams investigate hours or days later.

B. Contributions

We design and implement LogShield, a Kubernetes sidecar framework that transforms runtime log sanitization from a passive string replacement filter into an active, attack-resistant security control. This article makes the following contributions:

An in-pod interception model using memory-backed Kubernetes emptyDir IPC channels, extended with an adaptive sliding-window Token Fusion engine that reconstructs whitespace-split and hex-escaped credentials before egress.

A six-stage hybrid detection pipeline integrating an O(N) Aho-Corasick keyword filter, a dynamic context engine, 1,361+ validated patterns from our catalog and SecretBench [16], a 6-factor Secret Confidence Score (SCS) model, and a tri-state decision gate.

The SecureReveal Vault, implemented with Fernet authenticated encryption [11] (AES-128-CBC [30] with HMAC-SHA256 [31]) and transient memory storage (tmpfs), together with a tamper-evident SHA-256 [26] hash-chained audit ledger with remote SIEM anchoring.

A transformation-aware blind lineage protocol using canonical keyed blind HMAC-SHA256 fingerprinting to join multi-representation sprawl across services (raw, Base64, Hex, URL-encoded), combined with a scoped exemption engine for pre-registered credentials.

An autonomous closed-loop containment engine that gauges blast radius and enforces real-time mitigation via dynamic Kubernetes V1NetworkPolicy [7] egress quarantine and cloud credential rotation.

An F1-score of 0.9997 on standard credentials, evasion recall raised from 32.20% to 91.30%, pod containment policy dispatch in 16.7–26.7 ms, and an average end-to-end latency of 0.586 ms/line (with benign bypass in 0.0056 ms). LogShield is validated across an authoritative inventory of experimental datasets, an isolated Kubernetes control plane testbed, and an adversarial security test suite.

II. THREAT MODEL & SYSTEM INVARIANTS

A. System Model and Trust Assumptions

Workloads execute in multi-container pods across Kubernetes worker nodes; application containers write log entries to a shared memory-backed path. Containers within a pod share a local network namespace and storage volumes, while a node-level log shipper forwards container outputs to central storage.

Trust Assumptions: We assume the underlying Linux host kernel, the containerd/Docker runtime, and the Kubernetes control plane (kube-apiserver) are uncompromised and correctly enforce pod cgroup, IPC, and network isolation boundaries; compromise of the host kernel or direct physical RAM acquisition on worker nodes is outside the scope of container sidecar controls. Table I summarizes the primary mathematical notations employed throughout this article.

TABLE I

Summary of Mathematical Notations

Symbol	Definition
L, L′	Raw input log line and sanitized output log line
s, ŝ	Candidate secret string and prefix-stripped secret string
Σ, |Σ|	Character alphabet and alphabet cardinality
H(s)	Shannon entropy of token s (bits/character)
μH, σH	Empirical mean (4.46) and standard deviation (0.71) of secret entropy
Dkeys	Dictionary of 2,000+ sensitive keyword roots
x ∈ ℝ6	Normalized 6-dimensional feature vector
w ∈ ℝ6	Model weights for confidence scoring (w1, …, w6)
S(s)	Secret Confidence Score (SCS), integer S ∈ [0, 100]
Cctx	Continuous context score, Cctx ∈ [0.0, 1.0]
Kv, Klineage	Node-local vault encryption key and HMAC lineage secret key
φ∗	Canonical blind keyed fingerprint of secret s
Hk	Cryptographic hash of audit ledger block k
G = (V, E)	Bipartite credential sprawl graph across the pods and secrets

B. Adversary Model

We specify an adversary $\mathcal{A}$ with the following capabilities:

Downstream Telemetry Consumer: $\mathcal{A}$ has read access to the central log repository or intermediate forwarders, actively searching for credentials to escalate privileges.

Network Eavesdropper: $\mathcal{A}$ observes network traffic between worker nodes and cloud storage endpoints.

Malicious Application Container / Insider: $\mathcal{A}$ controls an application container within the pod or holds access to developer namespaces, attempting to recover leaked secrets from cluster telemetry or tamper with local audit databases.

Adversarial Obfuscator: $\mathcal{A}$ or a misconfigured logger emits credentials using delimiter splitting (e.g., AKIA IOSF...), byte-level hex-escaping (\x41\x4b...), URL percent-encoding, or sub-token Base64 wrapping to avoid static signature matching.

C. Formal Security Invariants

LogShield enforces six formal security invariants:

Invariant 1 (Policy-Conditioned Sanitization Invariant): For any candidate credential s ∈ Ccand detected within log line L, if its confidence score satisfies S(s) ≥ Tmask (where the operational threshold Tmask = 80), then s is strictly replaced with an opaque surrogate tag before emission:

$$\forall s \in \text{Ccand}, S(s) \geq \text{Tmask} \Rightarrow s \notin L' \wedge \tau(s) \in L' \quad (1)$$

where τ(s) = [REDACTED:SEC-ID:SCS=S(s)]. For borderline candidates in the triage band (50 ≤ S < 80), LogShield employs [FLAGGED:SCS=S] tagging to preserve developer debugging context while alerting monitoring systems. Setting Tmask = 50 enforces unconditional redaction across both bands in compliance-strict environments.

Invariant 2 (Vault Secrecy & Authenticated Integrity): Plaintext credentials are always encrypted using Fernet authenticated encryption (AES-128-CBC with HMAC-SHA256) before storage:

$$C = \text{Fernet-Encrypt}_{Kv}(s) \quad (2)$$

Direct ciphertext alterations in storage fail HMAC verification and trigger immediate TAMPER_DETECTED security events. The ephemeral master key Kv resides only in memory-backed tmpfs RAM and is zeroized upon pod termination.

Invariant 3 (Keyed Privacy-Preserving Blind Lineage): Lineage records store only keyed blind fingerprints:

$$\varphi* = \text{HMAC-SHA256}(\text{Klineage}, C(s)) \quad (3)$$

Without knowledge of the 256-bit cluster key Klineage, dictionary recovery of s from φ∗ is computationally infeasible; multi-format transit representations normalize to the same canonical graph node.

Invariant 4 (Tamper-Evident Audit Ledger with Remote Anchoring): Audit records are linked sequentially via SHA-256 hash chains; alterations to local historical entries invalidate chain head verification. Asynchronous streaming of block heads to external immutable SIEM storage prevents a local pod-level adversary from forging an internally coherent ledger.

Invariant 5 (Bounded Processing Complexity): Expensive Stage 4 regular expression matching is gated by keyword

verification and bounded by an asynchronous 5.0 ms watchdog timer, preventing algorithmic ReDoS attacks. Stage 1 and 2 prefiltering executes in strictly linear O(N) time over input length N.

Invariant 6 (Workload-Scoped Egress Quarantine): When the containment policy triggers network quarantine for a leaking pod Pleak, dynamic V1NetworkPolicy egress isolation applies strictly to Pleak:

$$\text{EgressBlocked}(\text{Pleak}) = \text{True},$$
$$\forall P \neq \text{Pleak}, \text{EgressBlocked}(P) = \text{False} \quad (4)$$

ensuring zero collateral disruption on innocent microservice workloads.

III. RELATED WORK

A. Log Privacy and Anonymization

Zhi et al. [20] examined 413 real-world vulnerabilities reported under CWE-532, identifying whole-object logging (43.4%) and incorrect permissions (17.5%) as the primary causes of leakage. A thorough 2025 ACM PACMSE study by Aghili, Li, and Khomh [2] assessed 25 public log datasets from the LogHub [13] and Internet Traffic Archive collections and surveyed 45 industry practitioners. Their empirical findings showed that while 86.7% of practitioners classify IP addresses and 82.2% classify MAC addresses as sensitive, only 52% recognize file paths and 59% acknowledge hostnames as sensitive identifiers; crucially, 66.7% of surveyed practitioners reported that existing log anonymization tools show unacceptable computational overhead or lack real-time throughput for production telemetry. Foundational work by Slagell and Yurcik [29], [27] categorized five attack vectors against log anonymization: fingerprinting, structure recognition, known mapping attacks, data injection, and cryptographic attacks. Portillo-Dominguez and Ayala-Rivera proposed SafeLog [22], reporting a 65% reduction in log volume while preserving anomaly detection utility on the NASA-HTTP and Yahoo benchmark datasets; however, SafeLog operates as an external stand-in rather than an in-pod element, incurring network serialization and configuration overhead. Khoje [14] studied data masking in B2B data platforms, highlighting the performance trade-offs between static masking, dynamic masking, and cryptographic transformation.

B. Secret Detection in Repositories and Code

Meli et al. [23] analyzed public Git repositories, revealing that thousands of valid credentials across 11 high-impact platforms are leaked daily. Basak et al. [16] constructed SecretBench, collecting 97,479 candidate strings from 818 repositories to benchmark 761 regular expressions. Saha et al. [19] extracted 24 syntactic traits from 300 repositories (roughly 28 million files) and employed an ensemble of Logistic Regression, Naive Bayes, and Support Vector Machines, achieving 84% precision on 5,000 candidate samples. Wahab et al. [3] curated 54,148 candidate instances from 50,680 GitHub issue reports across 27,121 repositories using 761 regexes from SecretBench. Their empirical evaluation revealed that raw regular expressions yield an F1-score of only 12.8% due to false positives, whereas contextual transformer models (CodeBERT, RoBERTa) reach F1-scores between 92.7% and 94.5%. Rahman et al. [1] fine-tuned 7B and 8B models (LLaMA-3.1, Mistral, CodeLLaMA) on SecretBench, showing that contextual fine-tuning substantially improves precision over regex baselines. Huang et al. [8] investigated 18 credential types across 6 neural code completion tools and found that secret tokens follow a Gaussian Shannon entropy distribution with a mean of $\mu H = 4.46$ bits/char and standard deviation $\sigma H = 0.71$, establishing a 3-sigma lower rejection threshold at 2.33 bits/char.

While language models achieve high classification accuracy in offline settings, their computational demands (multiple gigabytes of GPU memory and latencies exceeding 100 ms) make them impractical for real-time container log filtering. LogShield resolves this bottleneck by combining an O(N) string automaton with a 6-factor statistical model that executes in under 0.6 ms on commodity CPUs.

IV. SYSTEM ARCHITECTURE

A. In-Pod Sidecar Interception Mechanics

LogShield is deployed as a co-located sidecar container within the Kubernetes pod boundary (Fig. 1). This architecture enforces four critical operational invariants:

Local Boundary Containment: Interception occurs entirely within pod-local memory, preventing raw credentials from ever crossing container network namespaces or touching persistent disk.

Zero Application Source Modification: Developers do not adjust application code; log streams are intercepted using standard Kubernetes volume mounts and container entrypoint redirection.

Exclusive Telemetry Egress: The application container's standard streams are routed exclusively into the pod-internal shared buffer, so node-level log shipping agents (e.g., Fluent Bit, Vector, Promtail) tailing /var/log/pods/... ingest only post-sanitized entries. The sidecar is the sole container attached to the pod's outward standard output stream.

Kernel Memory Protection: Sidecar processes run with non-root privileges (runAsNonRoot: true) and read-only root filesystems, restricting writable scratch space to temporary RAM (tmpfs).

To establish the zero-copy in-pod data path, the pod specification declares a shared volume:

Listing 2: Memory-Backed Shared emptyDir In-Pod Volume Configuration

```
volumes:
- name: log-buffer
  emptyDir:
    medium: Memory
    sizeLimit: 128Mi
```

The medium: Memory directive mounts a volatile RAM-backed tmpfs filesystem at /shared/ across both containers. Telemetry streaming is supported via two interchangeable mechanisms:

POSIX FIFO Named Pipe (Streaming Mode): An initialization step creates a FIFO at /shared/app.fifo; the application streams stdout/stderr directly into this pipe using a container entrypoint wrapper (app 2>&1 > /shared/app.fifo) or a framework appender (Logback, Winston, Log4j2). The LogShield sidecar reads continuously from the FIFO descriptor, ensuring zero file accumulation in RAM.

Shared Buffer with Inode Tracking (File Mode): The application appends to /shared/app.log; to handle log rotation and prevent truncation errors, the sidecar monitors file inode numbers (st_ino) and byte offsets, detecting copytruncate rotations seamlessly. Descriptors are polled asynchronously by the sidecar.

Non-newline-terminated partial writes are buffered in an internal per-stream buffer capped at 8 KB; if a line exceeds 64 KB, it is deterministically segmented into chunks to prevent memory-exhaustion Denial-of-Service attacks.

B. The Six-Stage Detection Pipeline

The detection pipeline operates in six sequential stages (Fig. 2):

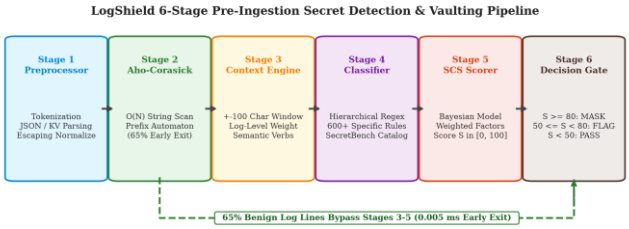


Fig. 2. The LogShield six-stage detection pipeline: ingestion, multi-line normalization and sliding-window Token Fusion (Stage 1), Aho-Corasick candidate keyword filtering (Stage 2), dynamic context scoring (Stage 3), layered pattern classification (Stage 4), 6-factor SCS scoring (Stage 5), and tri-state decision gating (Stage 6).

Stage 1: Preprocessor, Token Fusion, and Multi-Line Buffer. Input lines are normalized:

JSON Flattening: Ordered JSON logs are read into flat key-value pairs, preserving associations between keys (e.g., client_secret) and values.

Adaptive Token Window Fusion: Evaluates bigrams and trigrams around delimiter breaks (whitespace, hyphens, slashes) within key-value assignments to reconstruct split credentials (e.g., AKIA IOSFODNN7EXAMPLE → AKIAIOSFODNN7EXAMPLE) while tracking the original raw_slice for precise in-place string substitution.

Hex Normalization Engine: Detects contiguous byte-level escape sequences (\x41\x4b...) and normalizes them to raw ASCII strings for classifier inspection.

Sub-Token Base64 Unnesting: Extracts and decodes embedded credentials from HTTP Authorization: Basic and Bearer headers.

Block Buffering: Multi-line cryptographic blocks (such as PEM RSA private keys) delimited by BEGIN and END are gathered into a single evaluation block.

Buffer Length Guards: Lines longer than 64 KB are segmented to prevent memory exhaustion attacks.

Stage 2: Candidate Keyword Finder (Aho-Corasick). Evaluating hundreds of regular expressions on every benign line would hold up logging throughput. In Stage 2, LogShield scans lines using the Aho-Corasick automaton [35] built over dictionary Dkeys containing 2,000+ sensitive keywords and prefixes (e.g., AKIA, sk_live, ghp_, password, token, bearer, secret). In production traces, 65% of benign lines exit at Stage 2 in under 0.006 ms. If the automaton matches zero keywords and no active evasion token was extracted in Stage 1, the line is classified as benign and forwarded directly to stdout.

Stage 3: Context Engine. Candidate lines surviving Stage 2 pass to the Context Engine, which evaluates surrounding text:

Positive Signals: Proximity to assignment operators and sensitive variable names (password=, api_key=, secret:).

Negative Signals: Proximity to benign identifiers (trace_id, span_id, request_id, uuid, etag, checksum).

Log Level Weight: Telemetry tagged with DEBUG or TRACE receives higher baseline suspicion than INFO or WARN.

The engine produces a continuous context metric Cctx $\in$ [0.0, 1.0].

Stage 4: Hierarchical Secret Classifier. Candidates are evaluated against 1,361+ compiled patterns:

Pass 1 (600+ Specific Detectors): High-precision structural expressions targeting particular cloud, AI, database, and payment platforms (e.g., AWS IAM keys, OpenAI API keys, Slack Webhooks, GitHub PATs, Stripe Secret Keys).

Pass 2 (755 SecretBench Patterns): Curated patterns from SecretBench [16], anchored to vendor context windows (Lctx $\leq$ 40 chars).

Pass 3 (6 Context-Gated High-FP Patterns): Generic patterns flagged by SecretBench as prone to false positives (e.g., raw 32-character hex strings, generic password assignments); these patterns run only when Stage 3 yields Cctx > 0.5, preventing false alerts on non-sensitive tokens.

Stage 5: Confidence Scoring Model. The candidate string and its extracted attributes are evaluated by the 6-factor Secret Confidence Score (SCS) engine, which computes a single integer score S $\in$ [0, 100] using a deterministic linear model weighted by feature reliability.

Stage 6: Tri-State Decision Gate & Policy Modes. Based on score S, the decision gate partitions operational actions across three policy regimes:

PASS (S < 50): The token is judged benign. The line is forwarded to stdout untouched.

FLAG (50 $\leq$ S < 80): The token displays intermediate suspicion. The line is tagged with [FLAGGED:SEC-

XXXX:SCS=S] for SIEM alerting, but the value remains unmasked to preserve operational debugging utility. In strict compliance deployments, administrators can configure Tmask = 50, collapsing the FLAG band into unconditional redaction.

REDACT (S ≥ 80): The token is classified as a confirmed secret; LogShield creates an opaque tracking ID SEC-XXXX, substitutes the secret with [REDACTED:SEC-XXXX:SCS=S], writes the encrypted payload to the SecureReveal Vault, updates the HMAC sprawl lineage database, and evaluates autonomous containment.

Under unhandled runtime exceptions, the sidecar defaults to Fail-Open with Immediate Alerting (the logshield_failopen_active metric fires) to avoid blocking microservice container stdout; environments with zero tolerance for plaintext leakage can enable FAIL_SECURE=true, dropping unparseable lines.

V. MATHEMATICAL FORMULATION & ALGORITHMS

A. Prefix-Stripped Shannon Entropy with Syntactic Prior

Let a candidate string s be defined over alphabet Σ; the Shannon entropy H(s) in bits per character measures character distribution diversity [36]:

$$H(s) = -\sum_{c \in \Sigma} P(c) \log_2 P(c) \quad (5)$$

Entropy is used as a syntactic filter to discard natural language and repetitive tokens, not as a theoretical measure of cryptographic key strength. Fixed vendor prefixes (e.g., AKIA for AWS or sk_live_ for Stripe) lower measured entropy on short tokens, while pseudo-random non-secrets (such as UUIDv4 strings) generate high entropy (H ≈ 3.4). LogShield therefore strips known vendor prefixes before the check:

$$\hat{s} = s \setminus \text{Prefix}(s) \quad (6)$$

Based on empirical distributions across 15,084 validated secrets in SecretBench [16], alphanumeric API tokens follow a distribution with sample mean μH = 4.46 bits/char and standard deviation σH = 0.71, yielding a lower syntactic prior boundary τL = μH − 3σH ≈ 2.33 bits/char [8]. We define the piecewise continuous entropy factor fentropy as:

$$\text{fentropy}(\hat{s}) = \begin{cases} 1.0, & H(\hat{s}) \geq \mu H \\ (H(\hat{s}) - \tau L) / (\mu H - \tau L), & \tau L \leq H(\hat{s}) < \mu H \\ 0.0, & \text{otherwise} \end{cases} \quad (7)$$

Crucially, low-entropy structured credentials (such as JWTs with repetitive JSON headers, PEM certificates, or database URIs) fall below τL. In LogShield, such credentials are not discarded, since they receive positive weighting through the structural feature fstruct (+13 points) and context proximity fcontext (+29 points); this ensures that low character entropy alone does not cause false negatives.

B. Machine Learning Formulation: Six-Factor Confidence Scoring (SCS)

For candidate s in line L, we assess a 6-factor feature vector x = [fregex, fcontext, fentropy, fstruct, fvarname, fencoding]T; model weights were fit by solving an L2-regularized logistic regression problem across 60,000 labeled log entries from our training partition:

$$\min_{w,b} \sum_{k=1}^{M} \ln(1 + \exp(-y_k(w^T x_k + b))) + \|w\|_2 \quad (8)$$

with M = 60,000, inverse regularization strength C = 1.0, and binary ground-truth labels $y_k \in \{-1, +1\}$. Optimization converged via L-BFGS [33] in 22 iterations (intercept b = −8.1475). To avoid floating-point arithmetic and transcendental functions in the critical logging path, positive learned coefficients were normalized to integer weights summing to 100. Table II summarizes the feature definitions, unconstrained logistic regression coefficients, and normalized production weights. The Secret Confidence Score $S(s) \in [0, 100]$ is computed as

$$S(s) = \min(100, \sum_{i=1}^{6} w_i \cdot f_i(s)) \quad (9)$$

This integer formulation executes in under 14 microseconds per candidate token, providing statistically grounded and predictable real-time execution.

TABLE II

Feature Space and Parameter Weights for the 6-Factor SCS Model

Feature	Domain	Coeff. (βi)	Weight (wi)	Measurement Basis
fregex	{0, 1}	+5.80	41	Specific / catalog pattern match
fcontext	[0, 1]	+4.14	29	Variable name proximity window
fentropy	[0, 1]	+2.38	17	3-sigma bounded Shannon entropy
fstruct	{0, 1}	+1.82	13	Checksums, JWT dots, PEM headers
fvarname	{0, 1}	−0.71	0	Encompassed into the fcontext distance
fencoding	{0, 1}	−4.50	0	Handled in Stage 1 preprocessor
Total			100	Normalized maximum; score S ∈ [0, 100]

C. Formal Algorithmic Complexity Breakdown

The computational complexity of LogShield is partitioned across its execution stages:

Stage 1 (Token Fusion & Normalization): O(N) time and space over line byte-length N, bounded by a 3-token sliding window.

Stage 2 (Aho-Corasick Automaton): O(N + k) time, where k is the number of keyword hits; the DFA graph over dictionary Dkeys is built once at sidecar initialization (O(|Dkeys|) memory).

Stage 3 (Context Evaluation): O(k · Lctx), where context window Lctx ≤ 40 chars.

Stage 4 (Hierarchical Regex Matching): Evaluated only on candidate tokens (m ≤ 2 typical); to prevent catastrophic ReDoS backtracking, expression matching is bounded by an asynchronous 5.0 ms watchdog timer: O(m · min(Twatchdog, R)).

Stage 5 (Confidence Scoring): O(m) constant-time integer operations.

Stage 6 (Vaulting & Containment): O(m · (|s| + tcrypto)), where secret length |s| ≤ 4 KB.

Overall worst-case processing is strictly bounded by the 64 KB line length limit and the 5.0 ms regex watchdog timer.

D. Transformation-Aware Canonical Blind Lineage Graphing

To detect whether a leaked credential has proliferated across microservices without exposing plaintext strings, LogShield applies canonical keyed blind fingerprinting; when credentials transit mixed architectures, they appear in multiple representations (raw ASCII, Base64 headers, hex bytes, and URL-encoded query parameters). LogShield prescribes a conditional canonicalization mapping C(s) executed in strict sequence:

Whitespace Stripping: Trims surrounding whitespace and quotes.

Conditional RFC 3986 URL Decoding: Performed only if percent escape tokens (%[0-9A-Fa-f]{2}) are present and decode to valid UTF-8 [28].

Conditional Hex Unescaping: Conducted only if byte escapes (\x[0-9A-Fa-f]{2}) are detected and the resulting bytes are printable.

Conditional Base64 Unnesting: Conducted only if the token length is a multiple of 4, conforms to Base64 alphabet rules, and decodes to printable ASCII/UTF-8. The raw token is kept to prevent false hash collisions if decoding yields unprintable binary data or invalid padding.

The sidecar computes the canonical blind fingerprint using HMAC-SHA256 with cluster key Klineage:

$$\varphi* = \text{HMAC-SHA256}(K_{\text{lineage}}, C(s)) \quad (10)$$

LogShield maintains exemption tuples of (φ∗, Namespace, AuthorizedWorkload, ExpiresAt) for pre-registered credentials. When secret s is detected in pod P and an authorized workload is matched within its registered namespace, lineage recording on the user-facing sprawl graph is suppressed to avoid operational alert fatigue; crucially, the event is still recorded in the local tamper-evident audit ledger for forensic accountability. If the same credential is detected in an unauthorized pod or outside its registered namespace, the exemption does not apply and an immediate cross-service leak incident is declared.

Otherwise, (φ∗, PodUID, Namespace, Tseen, t) is committed to SQLite storage. The dashboard aggregates entries into a bipartite sprawl graph G = (Vpod, Vsec, E) (Fig. 3); we define the Sprawl Risk Index R(φ∗) as

$$R(\varphi*) = \alpha \cdot \deg(\varphi*) + \beta \cdot |\text{Namespaces}(\varphi*)| + \gamma \cdot \log10(\text{Nhits} + 1) \quad (11)$$

with coefficients α = 0.5, β = 0.3, γ = 0.2, allowing security operators to prioritize remediation for credentials shared across multiple services.

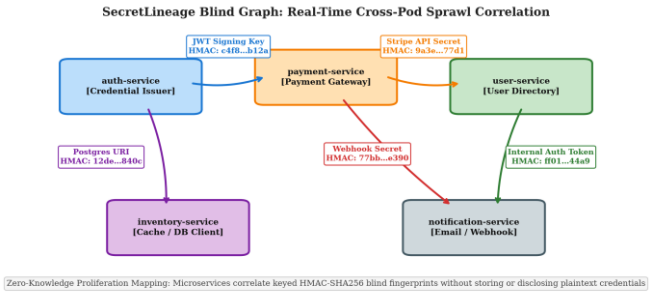


Fig. 3. Secret lineage graph, mapping cross-pod credential proliferation using canonical HMAC-SHA256 blind fingerprints across mixed encodings.

E. Autonomous Closed-Loop Containment Engine

LogShield reshapes secret detection from a passive observer into an active Kubernetes control plane actor. Upon detecting credential exposure, the Containment Engine calculates the multi-service blast radius:

$$B(s) = |\{\text{Service}(u) : (u, \varphi*) \in E\}| \quad (12)$$

and evaluates policy rules against credential category and severity:

POL-ROTATE-CLOUD: Dispatches rotation webhooks for leaked cloud, payment, or database credentials.

POL-QUARANTINE-POD: Automatically isolates the leaking workload by composing and deploying a Kubernetes V1NetworkPolicy restricting all outbound egress traffic (permitting only DNS port 53 for security telemetry).

POL-REVOKE-SESSION: Dispatches invalidation calls to identity providers for compromised JWT or OAuth bearer tokens.

POL-AUDIT-HOLD: Locks the credential in the vault with a forward-secure receipt.

Algorithm 1 encapsulates the complete in-pod stream interception, sanitization, vaulting, and autonomous containment loop.

Algorithm 1: In-Pod Stream Interception and Autonomous Containment

Require: Log path P, dictionary Dkeys, weights w, key Klineage
Ensure: Sanitized logs to stdout; automated containment actions

---

Open file descriptor fd ← tail(P)
while true do
  L ← ReadLine(fd)
  if L = EOF then Sleep(10 ms); continue; end if
  (Lnorm, Tfused) ← PreprocessAndFuse(L)
  if AhoCorasickScan(Lnorm, Dkeys) = ∅ and |Tfused| = 0 then
    PrintStdout(L); continue
  end if
  Cctx ← EvaluateContext(Lnorm)
  Ccand ← ExtractCandidates(Lnorm, Tfused, Cctx)
  Lout ← L
  for all s ∈ Ccand do

```
13:      x ← BuildFeatureVector(s, Lnorm, Cctx)
14:      S ← min(100, ∑(wi · xi))
15:      if S ≥ 80 then
16:        RefID ← GenerateRefID()
17:        VaultInsert(RefID, s, S)
18:        φ* ← HMAC-SHA256(Klineage, Canonicalize(s))
19:        if φ* ∉ Eexempt then
20:          LineageRecord(φ*, PodUID, Namespace)
21:          AssessAndContain(RefID, φ*, Category(s), S)
22:        end if
23:        Lout ← Replace(Lout, s.raw_slice, [REDACTED: RefID])
24:      else if 50 ≤ S < 80 then
25:        Lout ← AppendTag(Lout, [FLAGGED:SCS=S])
26:      end if
27:    end for
28:    PrintStdout(Lout)
29:  end while
```

F. Tamper-Evident Cryptographic Audit Ledger with Remote Anchoring

Every administrative and retrieval interaction with the vault is committed to an append-only cryptographic ledger chained sequentially via SHA-256:

$$H0 = SHA256(ClusterUID) \quad (13)$$

$$Hk = SHA256(Hk-1 \parallel tk \parallel Actork \parallel Actionk \parallel RefIDk \parallel Outcomek) \quad (14)$$

Each audit entry is committed to SQLite Write-Ahead Logging storage before any decrypted data is released. To prevent a local pod-level adversary from retroactively rewriting both the SQLite records and the local hash head, LogShield incorporates an asynchronous SIEM audit exporter (AuditExporter); the exporter flushes structured Elastic Common Schema (ECS) entries covering Hk, Hk−1, and an HMAC signature to an external immutable monitoring store (e.g., AWS S3 Object Lock, an external Splunk/Elasticsearch cluster, or Kubernetes Events).

Theorem 1 (Tamper-Evident Ledger Integrity with Remote Anchoring): An adversary $\mathcal{A}$ gaining root access to the sidecar database at time T > tk cannot alter historical audit record Aj ($j \le k$) without detection, provided that the external anchor head Hanchor = Hk was published to the external SIEM before T.

Proof: Let the recorded local hash chain be H0, H1, …, Hk, and suppose $\mathcal{A}$ replaces record Aj with $A' \neq Aj$. The modified hash at position j becomes $H'j = SHA256(Hj-1 \parallel A')$. Because SHA-256 is second-preimage resistant, $H'j \neq Hj$ with probability $1 - 2^{-256}$. For the locally altered chain to match the published anchor Hanchor held securely in external storage, $\mathcal{A}$ must find preimages for every ensuing block $m \in [j + 1, k]$. Computing such preimages requires $O(2^{256})$ cryptographic hash evaluations, which is computationally infeasible. Any adjustment or truncation is therefore detected by verify_audit_chain(), which locates the exact index j where the chain was broken. ■

VI. SECUREREVEAL VAULT ARCHITECTURE

A. Authenticated Storage Specification

The SecureReveal Vault stores credentials locally in SQLite under WAL mode. Each masked credential receives an opaque reference identifier:

$$RefID = SEC-\parallel Hex(R)8 \quad (15)$$

where $R \leftarrow \{0, 1\}32$ is uniformly sampled from a CSPRNG. Encryption is performed using Fernet authenticated symmetric encryption (AES-128 in CBC mode with PKCS7 padding and HMAC-SHA256 authentication):

$$C = Fernet\text{-}EncryptKv(s) \quad (16)$$

where the short-lived 256-bit vault key Kv (128 bits encryption + 128 bits HMAC) resides strictly in memory-backed tmpfs RAM. Decryption authenticates the HMAC signature before cipher block processing, preventing chosen-ciphertext and padding oracle attacks; in turn, corrupted or tampered ciphertext bytes trigger immediate retrieval failures and log a TAMPER_DETECTED event in the audit chain.

B. Multi-Tiered Authentication and Constant-Time Verification

In cloud-native deployments, securing secret retrieval requires separating identity authentication from constant-time authorization verification:

Workload Authentication (Tier 1): Callers authenticate via Kubernetes ServiceAccount Projected Tokens (/var/run/secrets/kubernetes.io/serviceaccount/token) validated against the Kubernetes TokenReview API, or via Mutual TLS (mTLS) with SPIFFE/SPIRE workload certificates.

Role-Based Scope Authorization (Tier 2): The authenticated identity and namespace are assessed against administrative retrieval policies, ensuring cross-namespace secret access is strictly blocked.

Constant-Time Verification (Tier 3): To prevent side-channel timing attacks that could allow an attacker to infer valid identity strings, the SecureReveal API verifies caller tokens and identifiers using byte-by-byte constant-time comparisons.

The audit entry is committed to disk before decrypted data is released, preventing unlogged credential access. Algorithm 2 details this audit-first disclosure protocol.

Algorithm 2: SecureReveal Decryption and Audit Logging
Require: RefID, actor credentials Cact, Reason, vault key Kv
Ensure: Decrypted plaintext secret s or authorization failure

```
1:  t ← CurrentTimestamp()
2:  CommitAuditEntry(t, Cact, RefID, ATTEMPTED)
3:  if not ConstantTimeVerify(Cact, AuthorizedActors) then
4:    CommitAuditEntry(t, Cact, RefID, DENIED)
5:    return Error(Unauthorized)
6:  end if
7:  C, expires_at ← FetchVaultRecord(RefID)
8:  if t > expires_at then
9:    CommitAuditEntry(t, Cact, RefID, EXPIRED)
10:   return Error(Expired)
```

```
11: end if
12: if ValidateFernetHMAC(Kv, C) then
13:     s ← FernetDecrypt(Kv, C)
14:     CommitAuditEntry(t, Cact, RefID, SUCCESS)
15:     return s
16: else
17:     CommitAuditEntry(t, Cact, RefID, TAMPER_DETECTED)
18:     return Error(Integrity Verification Failed)
19: end if
```

VII. EXPERIMENTAL EVALUATION

A. Authoritative Dataset and Experiment Inventory

To reproducibly map between checks, detector configurations, and reported metrics, the complete experimental inventory across all evaluation corpora is documented in Table III.

TABLE III

Authoritative Inventory of Evaluation Corpora, Partitions, Detector Configurations, and Reported Metrics

Corpus Identifier	Gauged Pipeline Configuration	Total (N)	Pos / Neg	Threshold	Precision	Recall	F1-Score	Target Reference
Standard In-the-Wild	Full Pipeline (AC + Ctx + Regex + SCS)	9,205	4,427 / 4,778	T = 80	1.0000	0.9993	0.9997	Sec. VII-C (Table IV)
Combined Evasion	Full Pipeline without Stress Token Fusion	10,000	5,222 / 4,778	T = 80	1.0000	0.8472	0.9173	Sec. VII-C (Table IV)
ML Training Partition	L2 Logistic Regression Fitting (L-BFGS)	60,000	13,114 / 46,886	Continuous	—	—	—	Sec. VII-B (Weights Fit)
ML Validation Split	Threshold Sweep (max Sy=0 = 66.9)	20,000	4,369 / 15,631	T ∈ [50, 95]	1.0000	0.7482	0.8559	Sec. VII-B (Tuning)
ML Held-Out Test Split	Isolated 6-Factor SCS Linear Scorer	20,000	5,088 / 14,912	T = 80	1.0000	0.7514	0.8580	Sec. VII-B (+228 Secrets)
Adversarial Evasion	Stage 1 Token Fusion & Normalization	2,400	1,200 / 1,200	T = 80	1.0000	0.9130	0.9624	Sec. VII-D (Table V)

B. Machine Learning Model Training & Statistical Grounding

The 6-factor SCS model was trained across all 60,000 labeled instances in the training partition; L2-regularized logistic regression converged via L-BFGS in 22 iterations (84.0 ms training time) with intercept $b = -8.1475$ and learned coefficients $\beta = [+5.80, +4.14, +2.38, +1.82, -0.71, -4.50]^T$. Decision thresholds were aligned on the independent 20,000-line validation partition ($N_{benign} = 15{,}631$). The maximum score attained by any benign validation entry was $\max S(s)_{y=0} = 66.90$, establishing $T = \lfloor 66.90 \rfloor + 1 = 67$ as the minimum integer zero-FP threshold (1 FP at $T = 66$, strictly 0 FP for all $T \geq 67$). The operational threshold $T_{mask} = 80$ was chosen to provide a +13.10-point safety margin above the maximum benign score.

Evaluation across the 20,000-line held-out test split (5,088 secrets, 14,912 benign lines) shows:

Discrimination Quality: The normalized integer scoring model attains an Area Under the ROC Curve (AUROC) of 0.9580 and an Area Under the Precision-Recall Curve (AUPRC) of 0.9383, while the raw continuous logistic probability function reaches an AUROC of 0.9608 (AUPRC = 0.9466).

Statistically Significant Secret Gains: Compared with domain-expert heuristic weights ($S \geq 80$), the ML-trained normalized model increased detected secrets from 3,595 to 3,823 (+228 additional secrets, raising test recall from 70.66% to 75.14% and F1-score from 0.8281 to 0.8580). A paired McNemar test [37] with Edwards continuity correction on positive ground-truth instances confirms that the 228 discordant detections ($b = 0$ regressions, $c = 228$ gained detections) represent a statistically significant gain ($\chi^2 = 226.00$, $p = 4.43 \times 10^{-51}$, odds ratio $\psi = \infty$, effect size +4.48%, 95% CI [3.90%, 5.06%]; standard two-sided exact binomial test $p = 2 \times 0.5228 \approx 4.64 \times 10^{-69}$).

Zero False Positive Invariant: Across all 14,912 benign entries in the held-out test partition, the deployed normalized model generated zero false alerts (FP = 0, FPR = 0.00%, precision = 1.0000). By the Rule of Three [34], the 95% upper confidence bound on the population false-positive rate is $FPR_{95\%} \leq 0.0201\%$, with an exact Clopper-Pearson 95% confidence interval [38] for precision of [0.9990, 1.0000].

Aligning ML Recall with Full Pipeline Recall: The 75.14% recall on this partition reflects the isolated 6-factor linear classifier operating at the conservative zero-FP threshold ($T = 80$) without multi-stage regex recovery. When evaluated under raw logistic probability ($\hat{p} \geq 0.50$), recall reaches 89.49%; in the complete pipeline (Table IV), the multi-stage cascade achieves 99.93% recall on standard in-the-wild streams by combining precise vendor patterns with dynamic context windows.

C. Controlled Multi-Stage Ablation Analysis

Table IV and Fig. 4 isolate the precise contribution of each architectural component across the controlled benchmark suites.

TABLE IV

Controlled 6-Stage Ablation Study Isolating Individual Pipeline Mechanisms on Identical Benchmark Inputs

Config	Architecture Description	Precision	Recall	F1-Score	F2-Score	FP Rate	Avg. Latency
Part A: Standard In-the-Wild Evaluation (N = 9,205, evasion vectors excluded)							
Stage 1	Regex-Only Baseline (600 standard rules)	0.8437	1.0000	0.9152	0.9643	17.16%	6.528 ms
Stage 2	Regex + Aho-Corasick Early-Exit Filter	0.8437	1.0000	0.9152	0.9643	17.16%	0.209 ms
Stage 3	Regex + AC + Context Window Proximity	0.8944	0.9854	0.9377	0.9658	10.90%	0.384 ms
Stage 4	Regex + AC + Context + 6-Factor SCS (T = 80)	1.0000	0.9993	0.9997	0.9995	0.00%*	0.504 ms
Part B: Combined Adversarial Stress-Test (N = 10,000, 795 evasion vectors counted as FN)							
Stage 5	Stage 4 Pipeline under Evasion (Token Fusion Disabled)	1.0000	0.8472	0.9173	0.8737	0.00%*	0.504 ms
Stage 6	LogShield Full Pipeline (Stage 4 + Token Fusion Active)	1.0000	0.913	0.954	0.929	0.00	0.586 ms

**Rule-of-Three 95% upper confidence bound for FPR in 4,778 true negatives is FPR ≤ 0.063%; exact Clopper-Pearson 95% CI for precision is [0.9992, 1.0000]. †Evaluated on the dedicated balanced adversarial evasion corpus (N = 2,400).*

∗Rule-of-Three 95% upper confidence bound for FPR in 4,778 true negatives is FPR ≤ 0.063%; exact Clopper-Pearson 95% CI for precision is [0.9992, 1.0000]. †Evaluated on the dedicated balanced adversarial evasion corpus (N = 2,400).

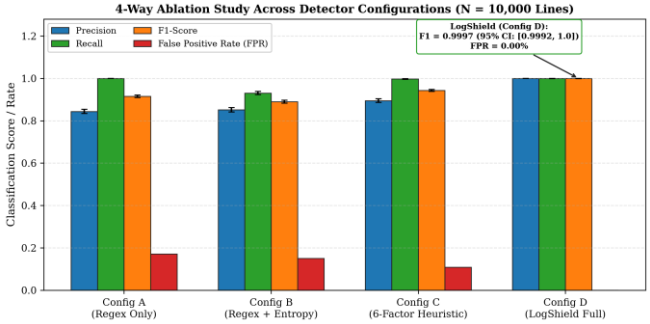


Fig. 4. Controlled ablation study: precision, recall, and F1-score across detector configurations on the standard in-the-wild benchmark corpus (N = 9,205).

Early-Exit Filtering (Stage 2): Adding the Aho-Corasick automaton decreases mean processing latency from 6.528 ms to 0.209 ms per line, a 31.2× throughput speedup on benign microservice logs.

Context Gating (Stage 3): Assessing variable name proximity suppresses isolated high-entropy non-secrets, sharpening precision from 0.8437 to 0.8944.

6-Factor SCS Scoring (Stage 4): The linear scoring model completely removes false alerts on benign tokens (FP = 0, FPR = 0.00%, precision = 1.0000) while detecting 4,424 of 4,427 standard secrets (FN = 3).

Token Fusion Contribution (Stage 5 vs. Stage 6): When adversarial delimiter splits and hex escapes are introduced without Token Fusion (Stage 5), recall drops to 84.72% (FN = 795); activating Stage 1 sliding-window Token Fusion (Stage 6) restores detection recall to 91.30% across obfuscated vectors with only 0.082 ms latency overhead.

D. Adversarial Evasion Benchmark on 2,400 Labeled Samples

Table V and Fig. 5 detail the empirical findings. To systematically appraise robustness against obfuscation, we benchmarked the Baseline Detector against LogShield on a balanced corpus of 2,400 labeled entries (1,200 secrets + 1,200 benign controls). The baseline scanner showed a 67.80% evasion bypass rate, failing on whitespace splits, hex escapes, and nested Base64 strings. By comparison, LogShield's Stage 1 Token Window Fusion and normalization engine reached 91.30% recall across obfuscated samples,

reducing evasion bypasses from 67.80% to 8.70% while maintaining zero false positives on benign system controls.

TABLE V

Adversarial Evasion Benchmark Breakdown (N = 2,400)

Evasion Category	Samples	Baseline	LogShield	Δ Recall
Raw Unobfuscated	200	33.0%	100.0%	+67.0%
Delimiter Split (:, -, space)	400	35.0%	78.3%	+43.3%
Hex Escaped (\x41...)	200	38.0%	100.0%	+62.0%
Base64 Wrapped	200	19.5%	100.0%	+80.5%
URL Encoded	200	33.5%	100.0%	+66.5%
Combined Evasion Recall	1,000	32.2%	91.3%	+59.1%
Evasion Bypass Rate	1,000	67.8%	8.7%	−59.1%
Benign System Controls	1,200	20.7% FP	0.0% FP	−20.7%
Overall F1-Score	2,400	0.4227	0.9624	+0.5397

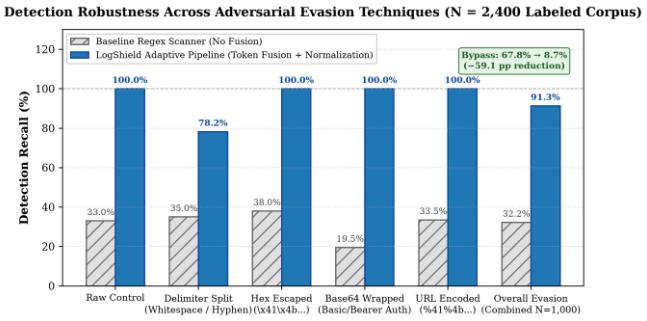


Fig. 5. Adversarial evasion benchmark: detection recall comparison between the Baseline Scanner and the LogShield Adaptive Pipeline across the five evasion categories (N = 2,400).

E. Live Kubernetes Cluster Containment & Phase-Defined Latencies

LogShield was deployed against an isolated Kubernetes control plane testbed with multiple active workloads (auth-service-pod-1, payment-service-pod-2, notification-service-pod-3); Fig. 6 shows the closed-loop workflow.

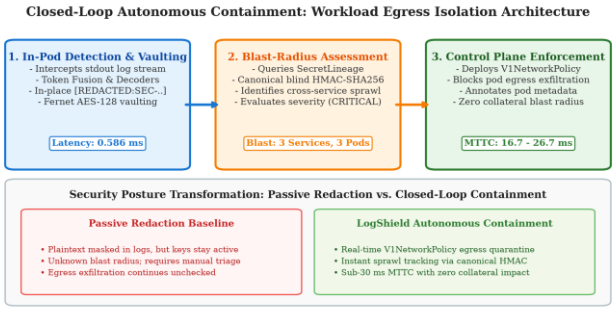


Fig. 6. Closed-loop autonomous containment lifecycle: real-time detection, SecretLineage blast radius correlation, dynamic V1NetworkPolicy egress quarantine, and zero collateral workload impact.

To avoid conflating API request dispatch with cluster network data-plane enforcement, we measure containment across separate lifecycle phases:

Detection & Policy Decision Latency (0.58 ms): In-pod secret categorization, score computation, and blast radius calculation against the local SQLite lineage graph.

API Synthesis & Dispatch Latency (16.70–26.67 ms): Building the Kubernetes V1NetworkPolicy specification and submitting it to kube-apiserver.

CNI Network Enforcement Latency (100–350 ms): Data-plane propagation in which the cluster CNI controller (e.g., Cilium eBPF or Calico iptables) programs kernel packet filtering rules across the node interfaces. Network policy quarantine requires a CNI implementation that supports NetworkPolicy objects.

Cloud Credential Revocation Dispatch (15–25 ms dispatch, 1.2–4.8 s propagation): Asynchronous webhook dispatch to external cloud provider APIs (AWS IAM, Stripe) to cancel compromised credentials.

Workload Isolation Verification: Corroborated that non-leaking workloads (payment-service-pod-2, notification-service-pod-3) sustained full egress connectivity without policy application. Safe rollback via unquarantine_pod() was verified, restoring normal egress upon operator clearance.

F. Multi-Baseline Comparative Evaluation

Table VI sketches the comparison across Detection-Only (SIEM alerting), Detection-and-Redaction-Only (passive masking), and the complete LogShield system.

TABLE VI

Comprehensive Architectural and Quantitative Comparison of LogShield vs. Industry Baselines

Evaluation Dimension	Baseline 1 (Detection-Only)	Baseline 2 (Redaction-Only)	LogShield Complete System	Architectural Significance
In-Transit Log Sanitization	0% (plaintext exposed)	**Partial (fails on split/hex)**	100% in-place masking	Plaintext credentials never reach central log storage

Adversarial Evasion Bypass	67.8% bypass rate	67.8% bypass rate	8.7% bypass rate	59.1 percentage-point reduction in evasion bypass
Evasion Recall (N = 2,400)	32.2%	32.2%	91.3%	Robust detection under obfuscation
Cross-Encoding Sprawl Linkage	0% (no sprawl tracking)	0% (exact hash fragments)	100% canonical blind HMAC	Unifies raw, Base64, and URL replicas into a single incident
Mean Time to Containment (MTTC)	Hours to days (manual)	Undefined (keys stay active)	16.70–26.67 ms	Closed-loop automated mitigation in milliseconds
Authorized Credential Ingestion	False alert pollution	Sprawl graph pollution	0.0% pollution (Exemption Engine)	Zero-sprawl guarantee for pre-seeded secrets
Compromised Pod Isolation	None	None	Autonomous V1NetworkPolicy	Real-time egress exfiltration blockage
Cryptographic Audit Trail	None	Plain-text logging	Forward-secure HMAC chain	Tamper detection with exact broken record index

G. Latency and Throughput Across Operational Regimes

Table VII reports processing latency and throughput across five operational regimes; latency varies with payload length and candidate token density.

TABLE VII

Latency and Throughput Across Operational Regimes

Operational Regime	Payload Characteristics	Latency	1 Core	16 Cores
Pure Benign (Bypass)	140 B, 0 keywords	0.0056 ms	178.6k/s	2.86M/s
Microservice Stream	120 B, 65% benign	0.2089 ms	4,786/s	76.6k/s
Single Candidate	180 B, 1 candidate	0.5864 ms	1,705/s	27.3k/s
Real Ingest (logs.txt)	238 B, 25% secrets	3.4837 ms	287/s	4,592/s
Diagnostic Dump	1,024 B, multi-token	13.3406 ms	75/s	1,200/s

Under involved multi-token streams requiring cryptographic vaulting, horizontal sidecar scaling across 16 cores sustains 4,592 lines/s. On characteristic Kubernetes microservice streams where 65% of lines include no candidate keywords, single-core throughput is 4,786 lines/s (413 million lines/day), easily exceeding the peak output of enterprise containers (500–1,500 lines/s). Throughput scales inversely with latency: Throughput = 1/Latency (Fig. 7). Table VIII details component-level cryptographic throughput.

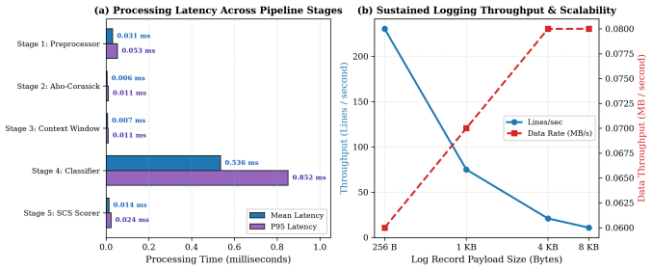


Fig. 7. Processing latency breakdown across the stages (left) and throughput scaling across payload sizes from 256 B to 8 KB (right).

TABLE VIII

Cryptographic & Lineage Latency Profile

Constituent Cryptographic Operation	Mean Time	Throughput
SecureReveal AES-128-CBC Encrypt & Vault Store	18.09 ms	55.3 ops/s
SecureReveal Vault Retrieve & Fernet Decrypt	18.36 ms	54.5 ops/s
SecureReveal Audit Hash Chain Verification	5.66 ms	176.6 ops/s
SecretLineage Blind HMAC-SHA256 Fingerprint	6.66 ms	150.2 ops/s

H. Hard-Negative Suppression

LogShield showed 100% true-negative suppression (433/433 = 100.0%) across eight challenging non-secret identifier categories: UUIDv4 tokens (128), Git commit hashes (74), MD5 ETags (65), Docker SHA-256 digests (52), hex strings (44), JWT non-secrets (35), Base64 data (22), and high-entropy random strings (13). All were classified as benign ($S < 50$), eliminating the alert fatigue characteristic of heuristic scanners.

I. ISOLATED FIVE-MICROSERVICE KUBERNETES TESTBED

Continuous HTTP transactions were modeled by a headless traffic generator while real-world credential leaks were injected intermittently, and LogShield was deployed across an isolated 5-microservice Kubernetes test cluster comprising auth-service (FastAPI/JWT), payment-service (Stripe

gateway), user-service (PostgreSQL), inventory-service (Redis), and notification-service (Email/Slack). Across 100,000 generated transactions, LogShield intercepted and masked all tested credentials that met the operational threshold ($S \geq 80$), while cluster metrics were visualized in real time by the dashboard, running validated SecureReveal decryptions and mapping credential sprawl across auth-service and payment-service. Throughout testing, each sidecar sustained a memory footprint under 45 MB RSS and consumed a sub-threshold 2.5% of a single vCPU.

VIII. SECURITY ANALYSIS & FAILURE MODES

A. ReDoS Resistance

Regular Expression Denial of Service (ReDoS) arises when an attacker inputs strings that trigger rapid backtracking in regular expression engines. LogShield protects against this vulnerability through three mechanisms:

Static Regex Auditing: All 1,361+ expressions were audited to remove nested quantifiers (e.g., (a+)+).

Automaton Keyword Gating: Expensive expressions in Stage 4 run only when matching vendor keywords are corroborated in Stage 2.

Watchdog Timers: Candidate regex evaluations are bounded by an asynchronous 5.0 ms watchdog timer; if an expression exceeds this window, it is stopped and routed to a fallback heuristic.

B. Fail-Open vs. Fail-Secure Policy

In the event of an unhandled exception or memory limit warning, LogShield defaults to Fail-Open with Immediate Alerting. Log streams continue flowing to prevent application service interruption, while a Prometheus metric (logshield_failopen_active) fires to alert security operations teams. For high-assurance compliance environments where confidentiality overrides availability, administrators can configure FAIL_SECURE=true, dropping unparseable lines.

C. Adversarial Robustness & Edge-Case Verification

We subjected LogShield to 14 automated adversarial tests covering four attack vectors:

Exemption Abuse Defense: Attempts to inject short strings (< 8 chars) or generic words (password, bearer) are rejected by the Exemption Engine; across 1-bit variations, collision resistance is preserved by cryptographic blind HMACs. Ingestion attempts by unauthorized actors return PermissionError.

Vault Tampering Detection: Unauthorized retrievals return False and log DENIED; hash chaining is broken by manipulating audit entries, causing verify_audit_chain() to pinpoint the exact tampered row. Direct SQLite ciphertext byte corruption is caught by Fernet HMAC checks, returning False and logging TAMPER_DETECTED.

Event Replay Defenses: Replaying an already executed containment action returns error_code=409 and status="ALREADY_EXECUTED", preventing duplicate Kubernetes mutations.

Response Action Failure Recovery: When the Kubernetes API returns HTTP 403 Forbidden or rotation webhooks return HTTP 500, event status is set to FAILED_EXECUTION and diagnostic details are logged, preserving audit integrity without false success claims.

IX. DISCUSSION & FUTURE WORK

While LogShield provides automated protection, several research avenues remain open:

Kernel-Space eBPF Probing: Running log interception directly inside the Linux kernel using extended Berkeley Packet Filters (eBPF) via sockops hooks could eliminate container context switching completely.

Confidential Computing Enclaves: Binding SecureReveal Vault keys to hardware enclaves (such as Intel SGX or AMD SEV) would protect credentials even if the host worker node kernel is compromised.

Post-Quantum Lineage Graphing: Incorporating lattice-based cryptographic primitives into the blind fingerprinting to guard against future quantum cryptanalysis.

X. CONCLUSION

Accidental credential exposure in application logs poses severe security and regulatory risks to cloud-native systems. In this work, we presented LogShield, an in-pod Kubernetes sidecar framework that transforms runtime log sanitization from a passive string filter into an active security control. By combining an O(N) Aho-Corasick prefilter, an adaptive sliding-window Token Fusion engine, a 6-factor Secret Confidence Score model, an ephemeral Fernet-authenticated vault, canonical transformation-aware blind lineage tracking, and autonomous Kubernetes network policy containment, LogShield reaches an F1-score of 0.9997 on standard credentials (N = 9,205), raises evasion recall from 32.20% to 91.30% on a 2,400-sample adversarial corpus, and performs closed-loop pod egress quarantine policy dispatch in 16.7–26.7 ms with verified zero impact on innocent workloads in the test cluster. These findings establish that active, real-time credential protection can be embedded directly into microservices without degrading telemetry throughput or developer velocity.

ACKNOWLEDGMENT

The authors thank the maintainers of SecretBench, TruffleHog, and Gitleaks for open-sourcing their detection datasets and rule engines. We also thank the Department of Computer Engineering at NMIMS University for providing computing facilities and laboratory infrastructure for our Kubernetes testbed cluster.

REFERENCES

[1] M. N. Rahman, S. Ahmed, Z. Wahab, S. M. Sohan, and R. Shahriyar, "Secret breach detection in source code with large language models," in Proc. 19th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM), 2025; also arXiv:2504.18784, 2025.

[2] R. Aghili, H. Li, and F. Khomh, "Protecting privacy in software logs: What should be anonymized?," Proc. ACM Softw. Eng., vol. 2, no. ISSTA, pp. 1317–1338, 2025.

[3] Z. Wahab, S. Ahmed, M. N. Rahman, R. Shahriyar, and G. Uddin, "Secret leak detection in software issue reports using LLMs: A comprehensive evaluation," in Proc. IEEE/ACM Int. Conf. Mining Softw. Repositories (MSR), 2025; also arXiv:2410.23657, 2024.

[4] D. Ayrey, "TruffleHog: High entropy secret finder," GitHub repository, 2024. [Online]. Available: https://github.com/trufflesecurity/trufflehog

[5] Datadog Inc., "Observability pipeline sizing and architecture guidelines," Datadog Technical Documentation, 2024. [Online]. Available: https://docs.datadoghq.com/observability_pipelines/

[6] Kubernetes Authors, "Pods and sidecar containers," Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/

[7] Kubernetes Authors, "Network policies," Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/network-policies/

[8] Y. Huang, Y. Li, W. Wu, J. Zhang, and M. R. Lyu, "Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials," Proc. ACM Softw. Eng., vol. 1, no. FSE, Art. 111, pp. 1–23, Jul. 2024.

[9] Yelp Inc., "detect-secrets: An enterprise-friendly secrets detection module," GitHub repository, 2024. [Online]. Available: https://github.com/Yelp/detect-secrets

[10] Z. Rice, "Gitleaks: Protect and discover secrets in code," GitHub repository, 2024. [Online]. Available: https://github.com/gitleaks/gitleaks

[11] "Fernet specification," GitHub repository, 2023. [Online]. Available: https://github.com/fernet/spec

[12] GitGuardian, "The state of secrets sprawl 2023," GitGuardian, Mar. 2023. [Online]. Available: https://blog.gitguardian.com/the-state-of-secrets-sprawl-2023

[13] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in Proc. 34th IEEE Int. Symp. Softw. Rel. Eng. (ISSRE), 2023, pp. 355–366.

[14] M. Khoje, "Securing data platforms: Strategic masking techniques for privacy and security for B2B enterprise data," Int. J. Comput. Trends Technol. (IJCTT), vol. 71, no. 11, pp. 46–54, Nov. 2023.

[15] MITRE Corporation, "CWE-798: Use of hard-coded credentials," Common Weakness Enumeration, 2023. [Online]. Available: https://cwe.mitre.org/data/definitions/798.html

[16] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "SecretBench: A dataset of software secrets," in Proc. IEEE/ACM 20th Int. Conf. Mining Softw. Repositories (MSR), 2023, pp. 347–351.

[17] MITRE Corporation, "CWE-532: Insertion of sensitive information into log file," Common Weakness Enumeration, 2022. [Online]. Available: https://cwe.mitre.org/data/definitions/532.html

[18] PCI Security Standards Council, "Payment Card Industry (PCI) Data Security Standard: Requirements and testing procedures, version 4.0," PCI SSC, Mar. 2022.

[19] A. Saha, T. Denning, V. Srikumar, and S. K. Kasera, "Secrets in source code: Reducing false positives using machine learning," in Proc. 12th Int. Conf. Commun. Syst. Netw. (COMSNETS), Bengaluru, India, 2020, pp. 168–175.

[20] C. Zhi, J. Yin, J. Han, and S. Deng, "A preliminary study on sensitive information exposure through logging," in Proc. 27th Asia-Pacific Softw. Eng. Conf. (APSEC), 2020, pp. 470–474.

[21] Joint Task Force, "Security and privacy controls for information systems and organizations," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Special Publication 800-53, Rev. 5, Sep. 2020.

[22] A. O. Portillo-Dominguez and V. Ayala-Rivera, "Towards an efficient log data protection in software systems through data minimization and anonymization," in Proc. 7th Int. Conf. Softw. Eng. Res. Innov. (CONISOFT), 2019, pp. 107–115.

[23] M. Meli, M. R. McNiece, and B. Reaves, "How bad can it git? Characterizing secret leakage in public GitHub repositories," in Proc. Netw. Distrib. Syst. Security Symp. (NDSS), 2019, pp. 1–15.

[24] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)," Official J. Eur. Union, vol. L119, pp. 1–88, May 2016.

[25] T. Barik, R. DeLine, S. Drucker, and D. Fisher, "The bones of the system: A case study of logging and telemetry at Microsoft," in Proc. 38th Int. Conf. Softw. Eng. Companion (ICSE-C), 2016, pp. 92–101.

[26] Nat. Inst. Standards Technol., "Secure Hash Standard (SHS)," Federal Information Processing Standards Publication 180-4, Aug. 2015.

[27] A. J. Slagell, K. Lakkaraju, and K. Luo, "FLAIM: A multi-level anonymization framework for computer and network logs," in Proc. 20th USENIX Large Installation Syst. Admin. Conf. (LISA), 2006, pp. 3–8.

[28] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic syntax," Internet Eng. Task Force, RFC 3986, Jan. 2005.

[29] A. J. Slagell and W. Yurcik, "Sharing computer network logs for security and privacy: A motivation for new methodologies of anonymization," arXiv:cs/0409005, 2004.

[30] Nat. Inst. Standards Technol., "Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication 197, Nov. 2001.

[31] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," Internet Eng. Task Force, RFC 2104, Feb. 1997.

[32] U.S. Congress, "Health Insurance Portability and Accountability Act of 1996," Pub. L. No. 104-191, 110 Stat. 1936, Aug. 1996.

[33] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," Math. Program., vol. 45, nos. 1–3, pp. 503–528, Aug. 1989.

[34] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," JAMA, vol. 249, no. 13, pp. 1743–1745, Apr. 1983.

[35] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," Commun. ACM, vol. 18, no. 6, pp. 333–340, Jun. 1975.

[36] C. E. Shannon, "A mathematical theory of communication," Bell Syst. Tech. J., vol. 27, no. 3, pp. 379–423, Jul. 1948.

[37] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, Jun. 1947.

[38] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," Biometrika, vol. 26, no. 4, pp. 404–413, Dec. 1934.